

Job Market Analytics Dashboard

Step 7 — Jobs Listing Page

Step 7 Overview

Status: ✓ Complete

Created a dedicated `/jobs` route and `jobs.html` template showing all 60 job records in a browsable listing with location and category filters. Each job card shows title, company, location, salary range, skill tags, and a direct link to the original job posting.

7.1 New Route Added to `app.py`

Added request to Flask imports:

```
from flask import Flask, render_template, request
```

New `/jobs` route:

```
@app.route("/jobs")
def jobs_page():
    category = request.args.get("category", "")
    location = request.args.get("location", "")
    query = supabase.table("jobs").select("*").order("scraped_at", desc=True)
    if category:
        query = query.ilike("category", f"%{category}%")
    if location:
        query = query.ilike("location", f"%{location}%")
    result = query.execute()
    jobs = result.data
    return render_template("jobs.html",
        jobs=jobs, category=category, location=location)
```

7.2 How Filtering Works

Part	What it does
<code>request.args.get("location", "")</code>	Reads <code>?location=london</code> from URL query string. Empty string if not provided.
<code>request.args.get("category", "")</code>	Reads <code>?category=IT</code> from URL query string.
<code>.order("scraped_at", desc=True)</code>	Returns most recently scraped jobs first
<code>.ilike("location", "%london%")</code>	Case-insensitive search — matches "London, UK", "Central London", etc.
<code>if category: query = query.ilike(...)</code>	Only adds filter if user typed something — empty string = no filter

7.3 `jobs.html` Features

Feature	How it works
Filter by location	Text input → GET form → <code>?location=value</code> in URL → <code>ilike</code> query
Filter by category	Text input → GET form → <code>?category=value</code> in URL → <code>ilike</code> query
Clear filters	Link to <code>/jobs</code> with no params — resets both filters
Job count	<code>{{ jobs length }}</code> — Jinja2 length filter shows "Showing 60 jobs"
Job cards	Loop over jobs, each card shows title/company/location/salary/skills/link
Skill tags	Each skill in <code>job.skills</code> rendered as a styled span badge
Back to Dashboard	Simple anchor link to <code>/</code>

7.4 Confirmed Output

URL	Result
/jobs	All 60 jobs shown, most recent first
/jobs?location=london	Filtered to London jobs only
/jobs?category=IT	Filtered to IT Jobs category
/jobs?location=central&category=IT	Both filters applied simultaneously

Result: ✓ Jobs listing page working with live filters and 60 job cards.

Build Progress

Step	Description	Status
1	Project setup — venv, packages, file structure, local Flask server	✓ Done
2	Adzuna API — register, get keys, write fetch_jobs.py, pull live data	✓ Done
3	Data cleaning — Pandas, normalize fields, extract skill tags	✓ Done
4	Supabase — create jobs table, store first batch of 60 jobs	✓ Done
5	Dashboard route — query Supabase, compute analytics, render template	✓ Done
6	Plotly charts — skills, location, salary interactive charts	✓ Done
7	Jobs listing page — /jobs route with location and category filters	✓ Done
8	UI polish — proper CSS, nav, better layout	Next →
9	GitHub + Railway deployment	
10	Add to portfolio via /admin	
11	(Stretch) ML salary prediction model	